

acadnet

```
dd if=/dev/zero of=subiect_$.i bs=1M count=12
```

```
tree -v
```

```
tree
```

```
head-c 10M /dev/urandom > database/public/info
```

sort -n = sortează **numeric crescător**

sort -nr = sortează **numeric descrescător**

◆ unzip arhiva.zip

dezarhivează efectiv (scoate fișierele)

◆ unzip -l arhiva.zip

NU dezarhivează

doar **listează conținutul**

```
cp /etc/skel/.bashrc ~/.bashrc
```

Apoi reîncarci configurația:

```
source ~/.bashrc
```

```
sudo chgrp -R tehnic public_data/scripts
```

Group Management in Linux

b) Creare grup și adăugare

✓ jobs -l Afișează și PID-ul fiecărui job

jobs -l ✓ jobs -p Afișează doar PID-urile

jobs -p ✓ jobs -r Afișează doar joburile în execuție (running)

jobs -r ✓ jobs -s Afișează doar joburile suspendate (stopped)

jobs -s

sudo renice -n 10 -p (PID)

2. Flow-ul de bază

git init # creezi repo

git add . # alegi fișierele

git commit -m "msg" # salvezi (commit)

3. Branch-uri

git branch # vezi branch-uri

git checkout -b nou # creezi + intri pe el

git checkout main # te întorci

git merge nou # unești

4. Verificare stare

git status

- ce ai modificat
- ce nu ai adăugat
- pe ce branch ești

5. Istoric

git log --oneline --graph --all

vezi toate commit-urile frumos

6. Modificări

```
git add nume_fisier  
git commit -m "ce ai făcut"
```

```
git branch -m nume_vechi nume_nou
```

rename

```
git branch -d nume_branch
```

STERGERe

ignoră majuscule – prinde "get", "GET", "Get"

```
grep -i "get" access.log
```

afișează numărul liniei

```
grep -n "" 4' access.log
```

```
2: 192.168.1.2 ... 401
```

```
4: 192.168.1.4 ... 403
```

liniile care NU sunt 4xx

```
grep -v "" 4' access.log
```

caută recursiv în tot directorul

```
grep -r "" 4' ./logs/
```

doar numele fișierelor care conțin erori 4xx

```
grep -rl "" 4' ./logs/
```

doar statusul găsit, nu toată linia

```
grep -oE "" 4'[0-9]{2}' access.log
```

```
" 401"
```

```
" 403"
```

```
" 404"
```

context – vezi ce s-a întâmplat în jurul erorii

```
grep -C 2 "" 4' access.log
```

linia de dinainte, linia cu eroarea, linia de după

Combinatii utile bash

câte erori 4xx per tip de status

```
grep -oE '4[0-9]{2}' access.log | sort | uniq -c
```

caută recursiv, afișează fișier + număr linie

```
grep -rn "" 4' ./logs/
```

caută cuvântul exact "GET" (nu "GETAWAY" etc.)

```
grep -w "GET" access.log
```

```
find . -type f -name "*.txt" -exec cp -t "Favourite characters" {} +  
  
find . -name "*.sql" | sort | xargs -l{} cp {} backend/db/archive/  
  
(# pentru fiecare linie primită, {} este înlocuit cu acea linie:  
  
# cp project/backend/users.sql backend/db/archive/  
  
# cp project/migrations/001_init.sql backend/db/archive/  
  
# ..)
```

1. find — căutare în sistem (live)

Ce face

Caută fișiere și directoare recursiv, în timp real.

Sintaxă

```
find [cale] [condiții]
```

Flaguri importante

```
-name "fisier"    # nume exact  
-iname "fisier"   # ignoră majusculele  
-type f          # fișiere  
-type d          # directoare  
-size +10M       # mai mari de 10MB  
-mtime -2        # modificate în ultimele 2 zile  
-empty           # fișiere goale  
-perm 777        # permisiuni exacte
```

Combinatii utile

Căutare + ștergere:

```
find . -name "*.tmp" -exec rm {} \;
```

Variantă mai eficientă:

```
find . -name "*.tmp" -exec rm {} +
```

Căutare + grep:

```
find . -type f -name "*.txt" -exec grep "parola" {} \;
```

Schimbare permisiuni:

```
find . -type f -name "*.sh" -exec chmod +x {} +
```

2. locate — căutare rapidă (bază de date)

Ce face

Caută în baza de date a sistemului, nu scanează live.

Sintaxă

```
locate nume
```

Flaguri

```
-i # ignoră majusculele  
-c # numără rezultate  
-n 10 # primele N rezultate  
-r # regex
```

Combinatii

```
locate -i config  
locate -c .txt  
locate -n 10 cache
```

Actualizare bază de date

```
sudo updatedb
```

3. fallocate — creare fișiere rapide

Ce face

Creează fișiere de o anumită dimensiune fără a scrie date reale.

Sintaxă

```
fallocate -l DIMENSIUNE nume_fisier
```

Flaguri

-l SIZE # dimensiune
-o OFFSET # începe de la un offset

Exemple

fallocate -l 100M test.bin
fallocate -l 1G file.iso
fallocate -o 1M -l 10M file.bin

4. type — identificare comenzi

Ce face

Arată ce este o comandă: alias, builtin sau executabil.

Sintaxă

type comanda

Flaguri

-a # toate locațiile
-t # tipul comenzii

Exemple

type ls
type -a python
type -t cd

Diferențe esențiale

Comandă	Funcție
find	căutare live în sistem
locate	căutare rapidă din bază de date
fallocate	creează fișiere de dimensiune fixă
type	identifică tipul unei comenzi

Explicație flaguri tar

tar -cjf logs_recent.tar.bz2 -T -

Flag	Semnificație
c	create — creează o arhivă nouă
j	bzip2 — comprimă cu algoritmul bzip2 (pentru .bz2)
f	file — urmează numele fișierului arhivă (mereu ultimul din grup, imediat urmat de numele arhivei)
-T -	files-from — citește lista fișierelor de arhivat dintr-un fișier; - înseamnă stdin (adică ce vine prin pipe de la find)

Dacă ai fi vrut gzip în loc de bzip2, puneai z în loc de j. Pentru xz puneai J (majusculă).

Alte flaguri comune tar

Flag	Semnificație
x	extract — dezarhivează
t	list — listează conținutul arhivei fără a extrage
v	verbose — afișează fișierele procesate
z	compresie gzip
J	compresie xz
C <dir>	schimbă directorul de lucru înainte de operație

pack:

```
@if [ -f acadnet2026.tar.gz ]; then \  
echo "Arhiva exista deja!"; \  
else \  
tar -czf acadnet2026.tar.gz Makefile *.c *.h); \  
fi
```

```
for i in {2..100..2} din 2 in 2
```

Ca să vezi valoarea nice a procesului cu PID 2102, ai câteva variante simple pe Linux:

Varianta 1 (cea mai directă)

```
ps -o nice= -p 2102
```

Sintaxa de bază:

```
* * * * * comanda
| | | | |
| | | | └─ ziua din săptămână (0-7, 0 și 7 = duminică)
| | | └── luna (1-12)
| | └─── ziua din lună (1-31)
| └─── ora (0-23)
└─── minutul (0-59)
```

Comenzi utile:

```
bash

crontab -e    # editează crontab-ul tău
crontab -l    # listează joburile existente
crontab -r    # șterge toate joburile
```



0 9 * * 1 echo "Sistem verificat" >> /tmp/audit.log

a) Script condiții meteo București Se poate folosi utilitarul curl împreună cu serviciul wttr.in.

Bash `#!/bin/bash curl -s wttr.in/Bucharest?format=3`

b) Script meteo cu argument

Bash `#!/bin/bash curl -s wttr.in/$1?format=3`

ex

(c) build: acadnet2025

```
gcc main.c firststep.c-o acadnet2025
```

```
gcc-c secondstep.c-o secondstep.o
```

```
gcc zeus.o secondstep.o-o sample
```

run: acadnet2025

```
./acadnet2025
```

clean:

```
rm secondstep.o sample acadnet2025
```

.PHONY: pack

pack:

```
tar-cvzf acadnet_rules.tar.gz *.c *.h Makefile
```